Figure 7.8 Highlighting the very different parts of what is usually considered just one platform

Leaving out parts of the interface is troublesome because if you change a part, you will cause serious value deterioration to the platform. For instance, if we decide to not use the Git repository anymore, teams Green and Yellow will have to redo a lot of their current data products. Analysts won't have them available for quite some time, and we will lose a lot of value over the transition period. If, instead, we keep the old parts of the interface open and slowly introduce additional functionality, we only increase the value of the platform.

Interfaces and why they are so hard to spot

The word *interface* is an overloaded term. In terms of platforms, an interface to a platform X is something really generic; it's "the ways of interacting with platform X."

From the platform perspective, an interface can be much more than a technically implemented REST interface. The central town square becomes a platform, with its interface being the location and the time.

And the same is true of a self-serve data platform. The interface might consist of practices and processes you've put into place to use, such as the central Git repository to upload CSVs.

All of these make up the interfaces, not just the technical parts. All of these are supposed to stay "mostly fixed." That's why they are so hard to spot and so important.

We can now easily point out the *platform kernel*—it's everything hidden behind the interfaces. The actual software running the DataHub and the data product registration script are definitely parts of the platform kernel that we can easily exchange in order to create more value.

And that is exactly what we are going to do in the next section. We will keep the interface fixed, and add a new part to the interface by making the kernel more modular and exposing more parts of it. Let's see that applied.

7.3.2 Platform architecture applied